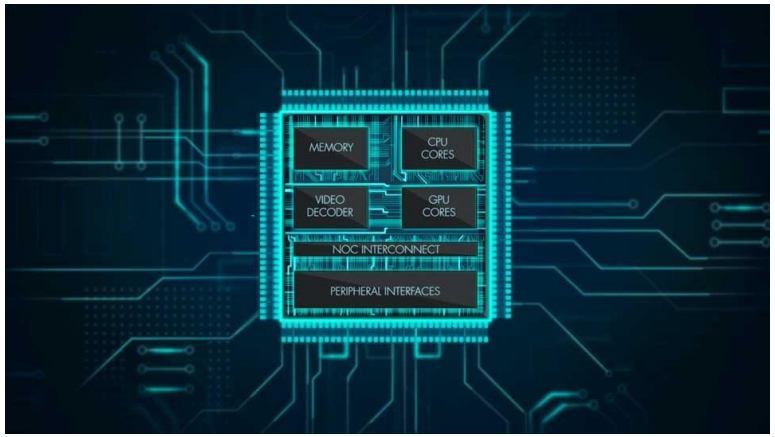
Digital System Design

Basic Operation of 1D Convolution



Date : 2026. 04. 16.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

Contents

- Conv1d
- Conv1d Computational Parallelism

Contents

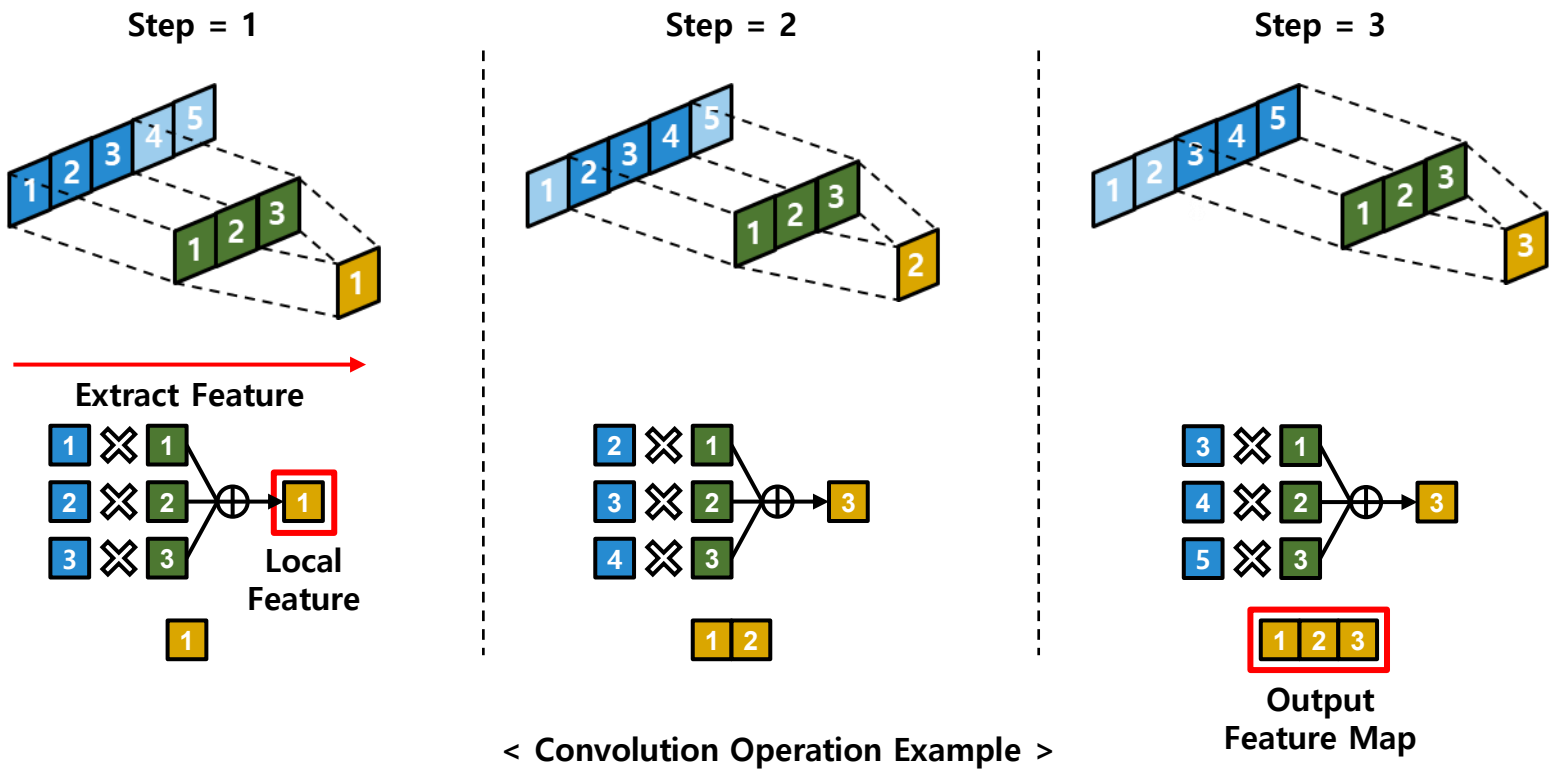
- Conv1d

- Conv1d Computational Parallelism

Concept of Conv1d

What is Convolution (Conv)?

- An operation that extracts **local features** from the input.
- Performs computation by **sliding a kernel (weights)** over local regions of the input.
- Each output element is computed as a **dot product between input and kernel.**
- Produces an output in the form of a **feature map.**
- This operation involves key concepts such as **sliding window**, **stride**, and **padding**.



< Convolution Operation Example >

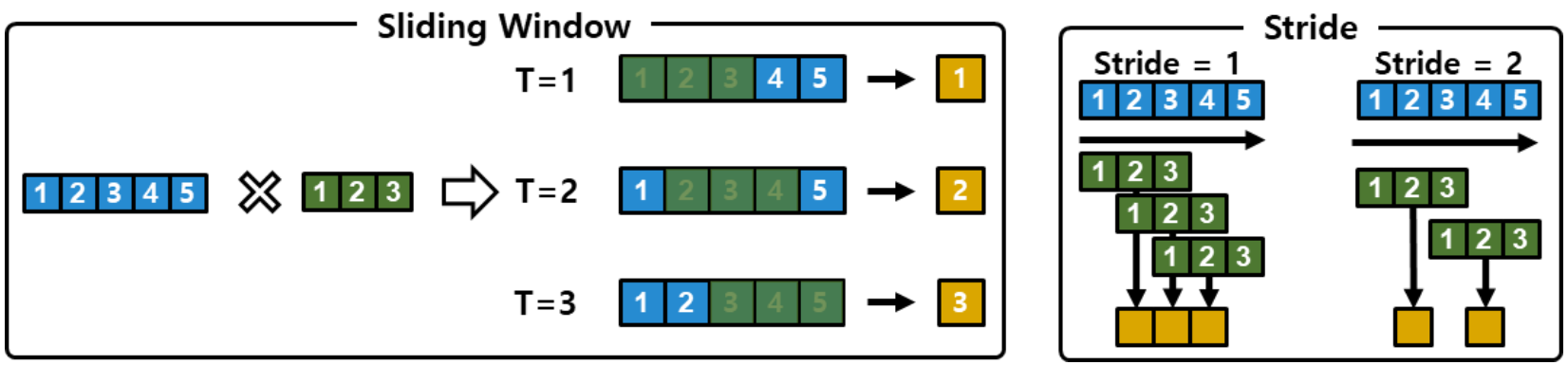
Sliding Window & Stride

Sliding Window

- Input and kernel perform dot product over a **local region.**
- The kernel **moves sequentially** across the inputs from **left-to-right** to generate outputs.

Stride

- Defines the **step size** of kernel movement.
- As stride increases, **number of operations & output size decreases.**
- Directly affects **computation, output resolution,** and **memory access.**



< Sliding Window and Stride Operation >

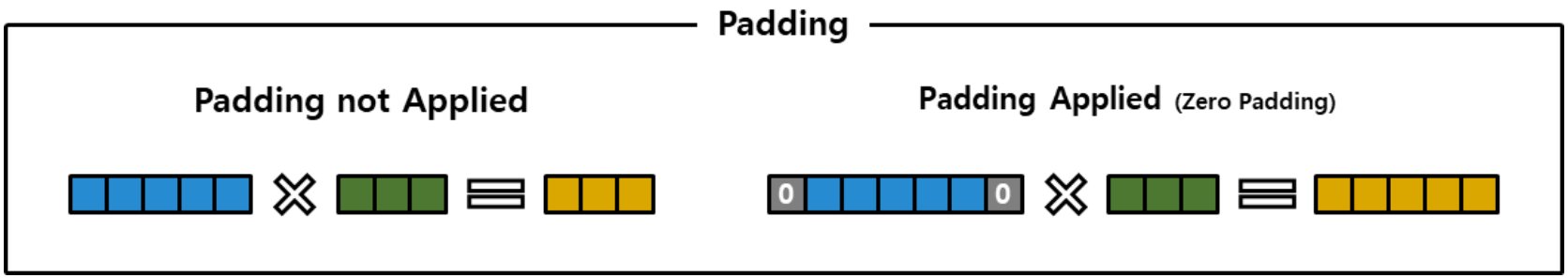
Padding

Padding

- Extends the input by adding values around the border.
  - Supports various padding types. (e.g., zero, reflect, replicate)
  - **Zero padding is most commonly used.** (also used in this work)
- Output size varies depending on whether padding is applied.
  - In hardware, zero padding is implemented via zero injection.

Role of Padding

- **Preserves spatial dimensions of the output.**
- Without padding, output size **decreases after convolution.**



< Padding Operation (Zero Padding) >

Computational process of Conv1d (1)

Convolution as Nested Loop Structure (1)

- Convolution can be viewed as repeated dot products.
- Its can be explicitly expressed using nested loops.
- **Loop (W)**: Slide the window horizontally.

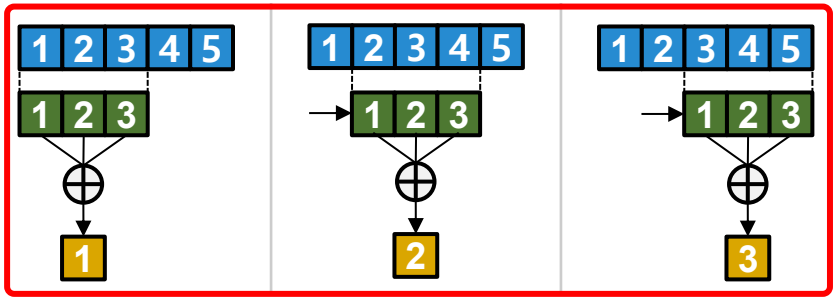
W: Width of Input
K: Width of Kernel
Q: Width of Output

Loop (Q)

```
for (int q = 0; q < Q; q++) {        // output width
    output[p][q] =
        input[q + 0] * kernel[0] +
        input[q + 1] * kernel[1] +
        input[q + 2] * kernel[2];
}
```

< Conv1d Algorithm-Level (Loop-based) >

Loop (Q)

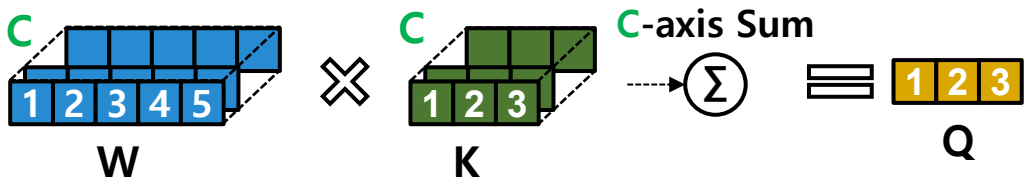


< Conv1d Operation-Level (MAC-based) >

Computational process of Conv1d (2)

Convolution as Nested Loop Structure (2)

- The input consists of multiple input channels (C). (e.g., RGB)
- Dot products across channels are summed. → **Reduction Sum**
- **Loop (C)**: Iterates over input channels. (1:1 input–kernel mapping)



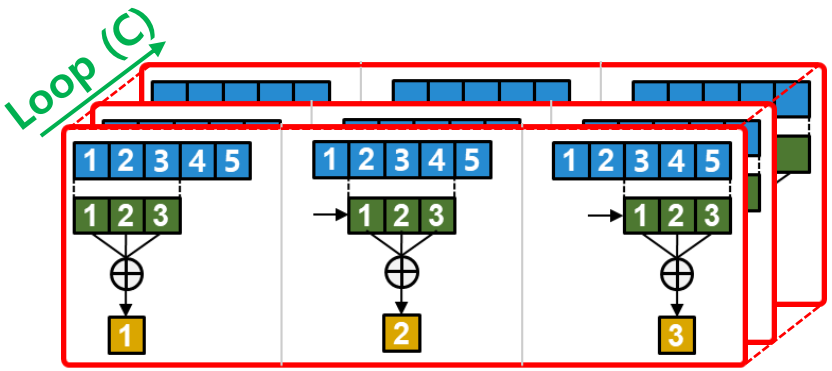
W: Width of Input
K: Width of Kernel
Q: Width of Output

C: # of Input Channels

```
for (int c = 0; c < C; c++) {            // input channel

    for (int q = 0; q < Q; q++) {        // output width
        output[q] +=     // Reduction Sum
            input[c][q + 0] * kernel[c][0] +
            input[c][q + 1] * kernel[c][1] +
            input[c][q + 2] * kernel[c][2];
    }
}
```

< Conv1d Algorithm-Level (Loop-based) >

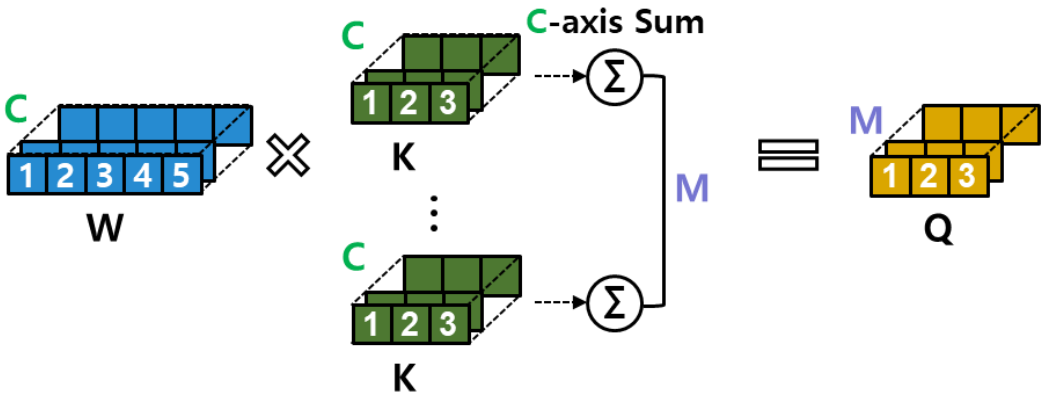


< Conv1d Operation-Level (MAC-based) >

Computational process of Conv1d (3)

Convolution as Nested Loop Structure (3)

- The output also consists of multiple output channels (K).
- Each output channel is generated by a **kernel group across channels**.
- **Loop (K)**: Iterates over output channels (kernel groups).

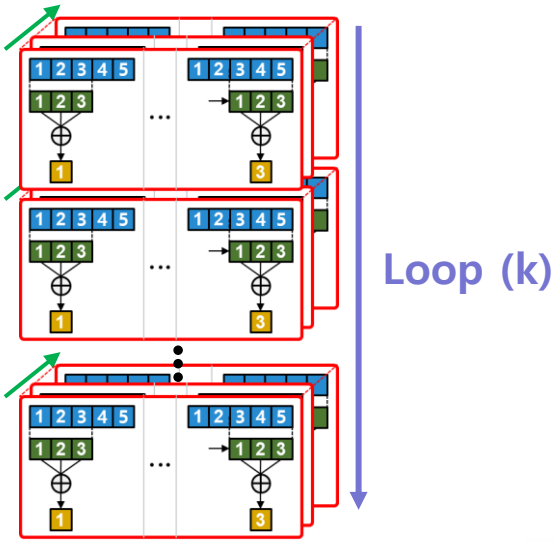


W: Width of Input
K: Width of Kernel
Q: Width of Output
C: # of Input Channels

M: # of Output Channels

```
for (int k = 0; k < K; k++) {                // output channel
    for (int c = 0; c < C; c++) {            // input channel
        for (int q = 0; q < Q; q++) {  // output width
            output[k][p][q] +=          // Reduction Sum
                input[c][p][q + 0] * kernel[k][c][0] +
                input[c][p][q + 1] * kernel[k][c][1] +
                input[c][p][q + 2] * kernel[k][c][2];
        }
    }
}
```

< Conv1d Algorithm-Level (Loop-based) >



< Conv1d Operation-Level (MAC-based) >

Contents

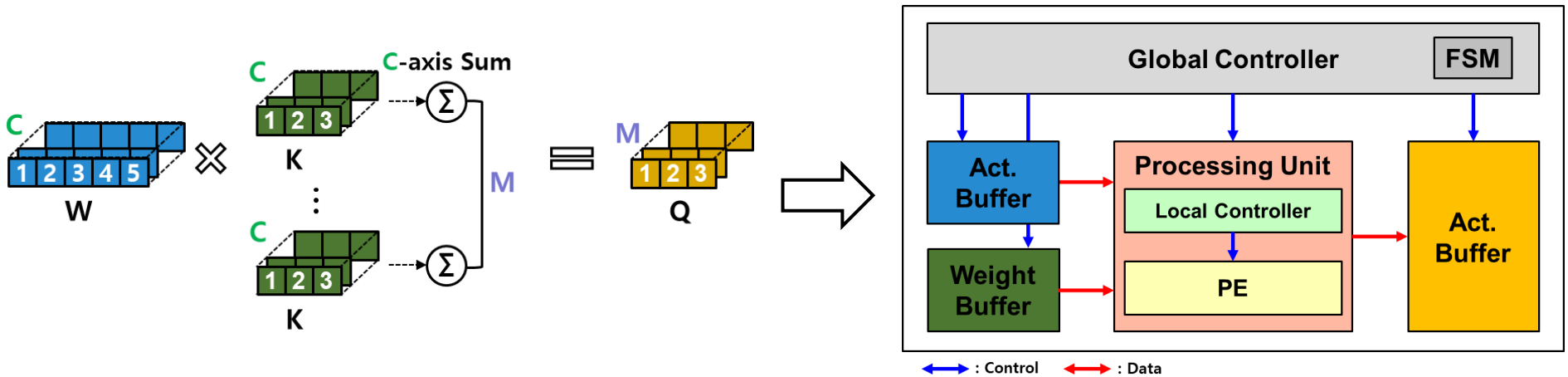
- Conv1d

- Conv1d Computational Parallelism

Computation Mapping for Conv1d

How is convolution mapped to hardware (RTL)?

- Conv1d is implemented through **repeated MAC operations** and **data movement.**
- Efficient hardware design requires considering both computation granularity and execution flow.
- **Computation Mapping**
  - **How to partition and map computations onto hardware.**
  - Should consider **Conv1d characteristics** (*e.g.,* kernel size, number of channels).
  - The architecture is determined based on **design goals (High Throughput vs. Low Resource).**
  - Directly impacts **parallelism, latency,** and **resource utilization.**

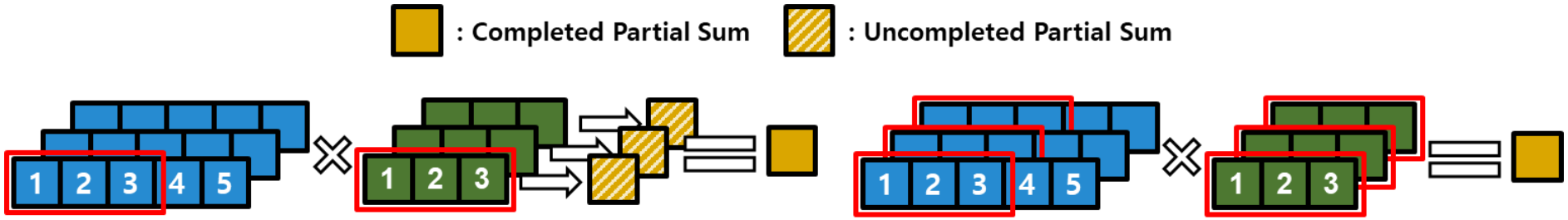


< Conv1d to Hardware Mapping (Controller, Buffer, PE) >

Computation Granularity

Computation can be partitioned at different levels

- **Kernel-level (dot-product)**
  - Computes **one dot product (one kernel),** matching the kernel size.
  - Provides an intuitive mapping between computation and hardware.
  - Requires **channel-wise reduction at output generation,** requiring accumulation registers and control logic.
- **Channel + Kernel-level**
  - Computes **multiple channels in parallel,** enabling **high parallelism.**
  - Allows channel-wise reduction within a single cycle.
  - **Less flexible when channel configurations change (utilization loss).**
  - Requires more complex PU connections and additional hardware resources.



< Example: Kernel-level Parallel Compute >

< Example: Channel × Kernel Parallel Compute>

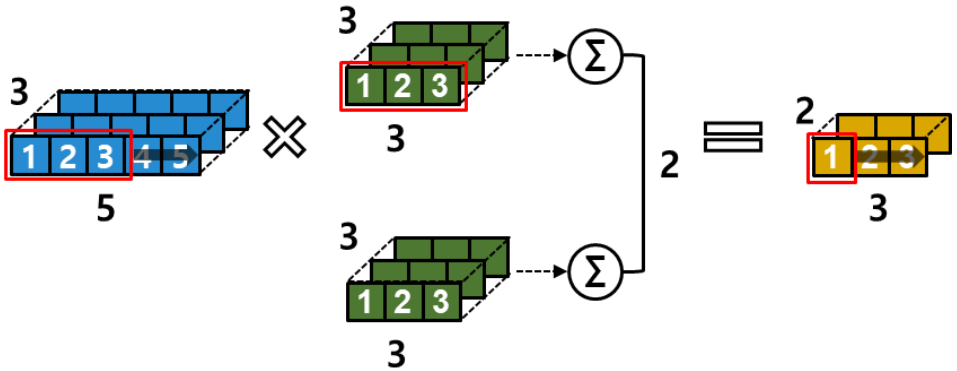
Design Trade-off in PE Parallelism

Throughput–utilization trade-off

- **Fewer PEs : 3 PEs (Kernel-Level Parallelism)**
  - Lower resource usage
  - Lower throughput

Number of PEs	:	**3 Units ↑**
Compute per Output	:	9 (Channel x Kernel)
MAC per Cycle	:	3 (PEs)
Cycle per Output	:	**3 Cycles ↓**

< Parallelism vs. Execution Cycles >

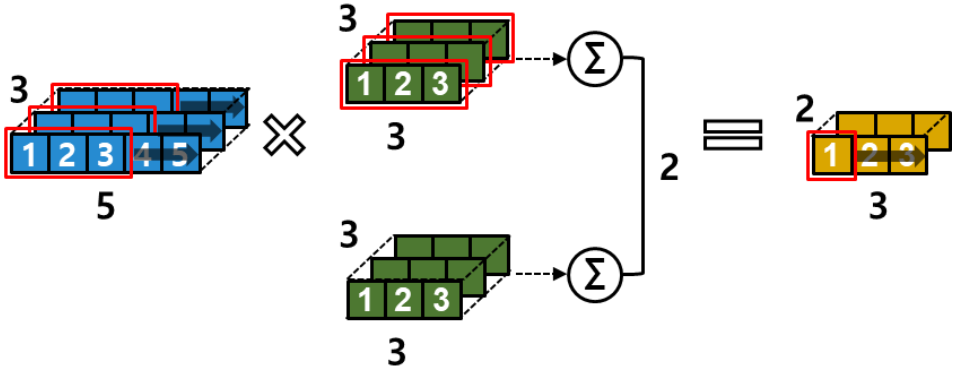


< Kernel-level Parallel Conv1d using 3 PEs >

- **More PEs : 9 PEs (Channel × Kernel Parallelism)**
  - Higher throughput
  - Higher resource usage

Number of PEs	:	**9 Units ↑**
Compute per Output	:	9 (Channel x Kernel)
MAC per Cycle	:	9 (PEs)
Cycle per Output	:	**1 Cycles ↓**

< Parallelism vs. Execution Cycles >

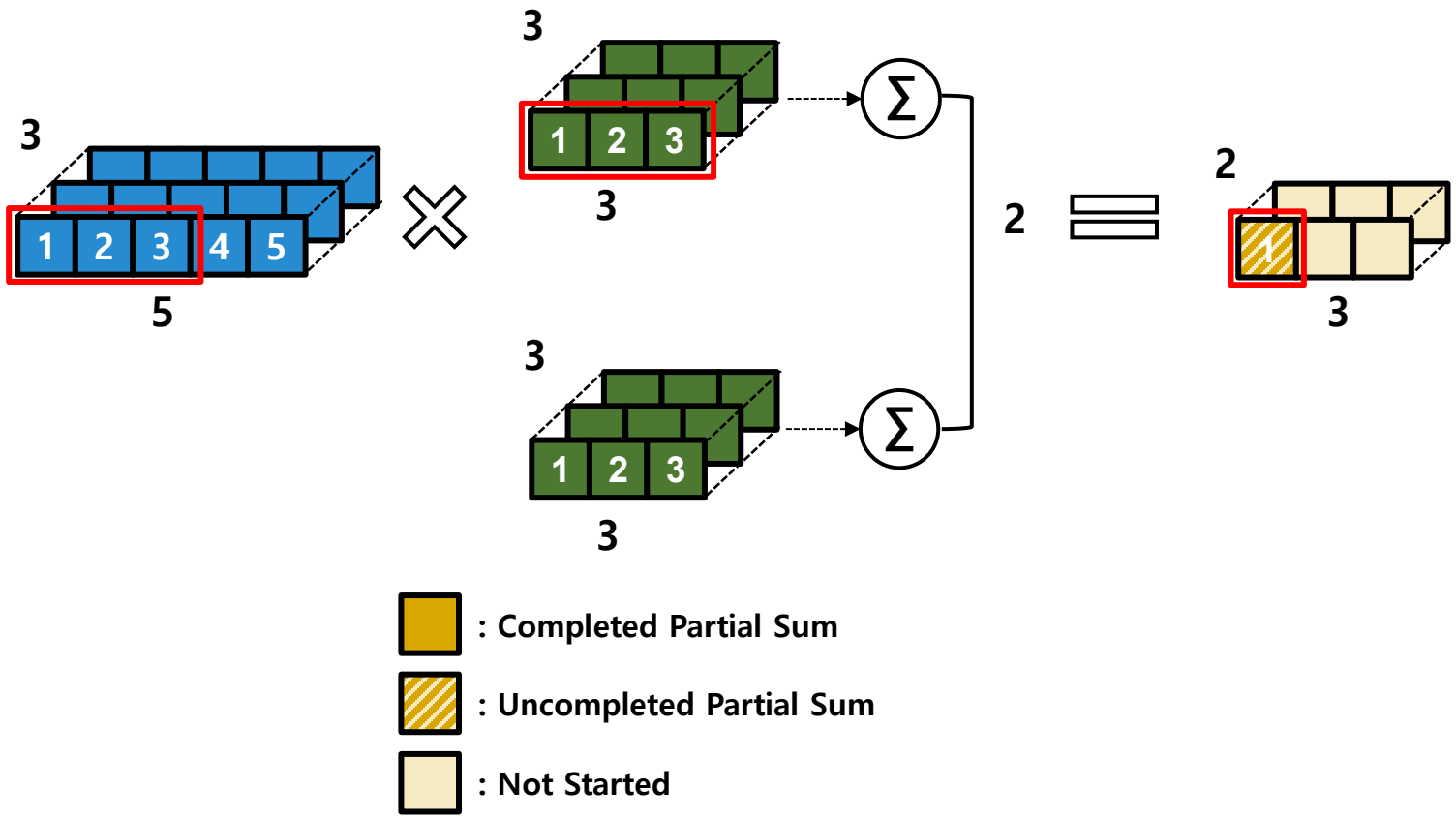


< Channel × Kernel Parallel Conv1d using 9 PEs >

Conv1d Execution Flow (1)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

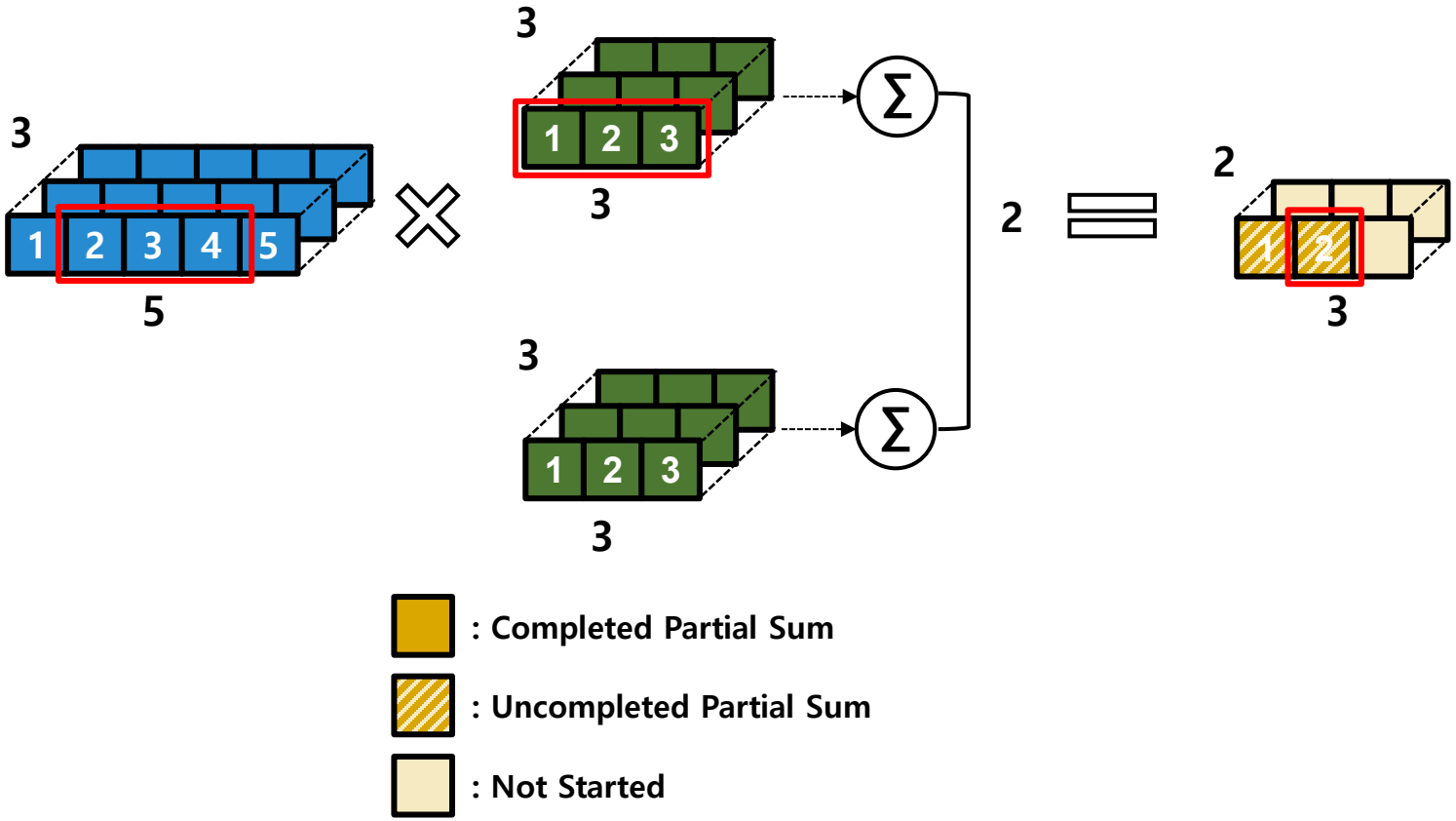


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (2)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

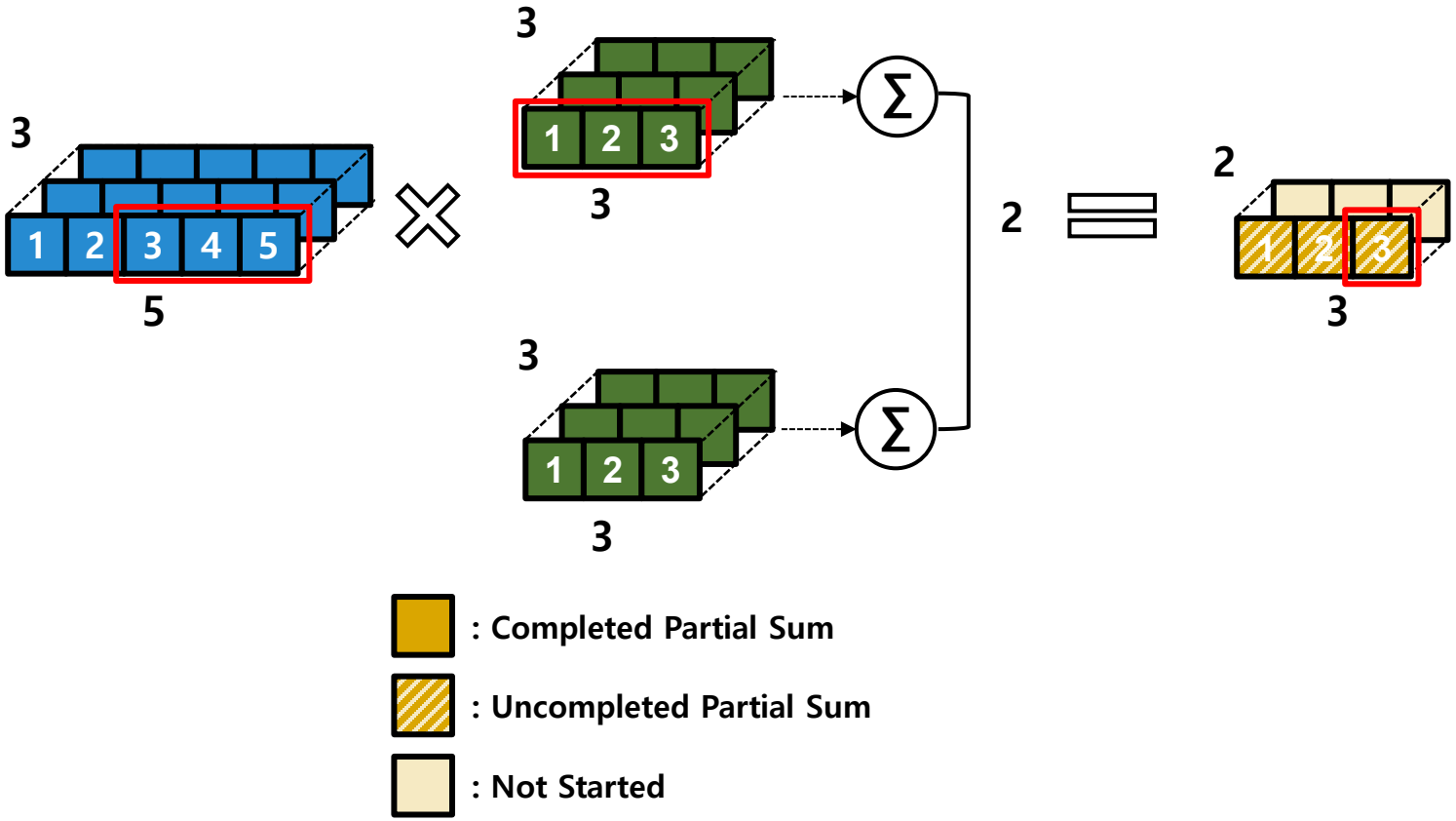


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (3)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

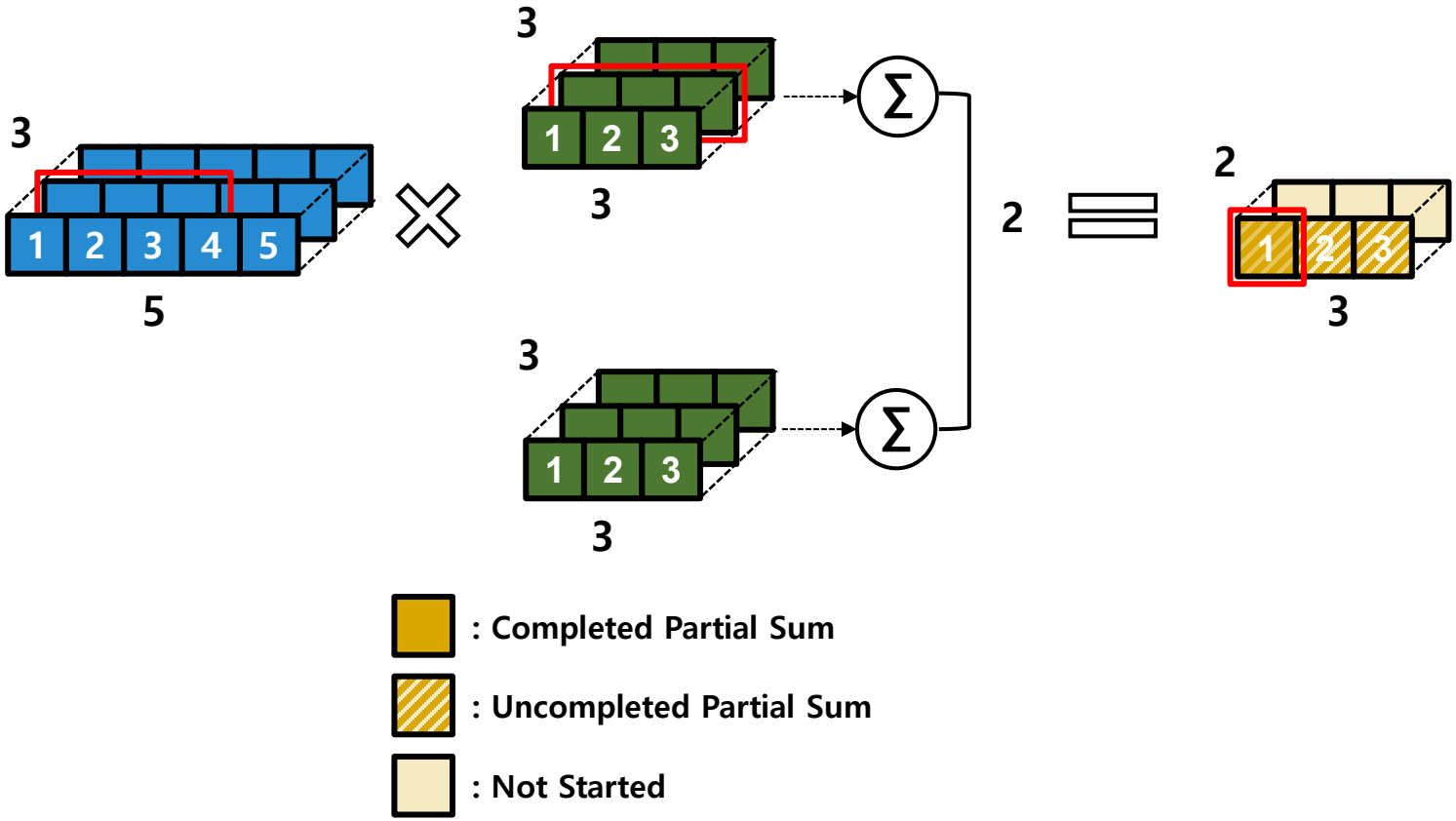


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (4)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

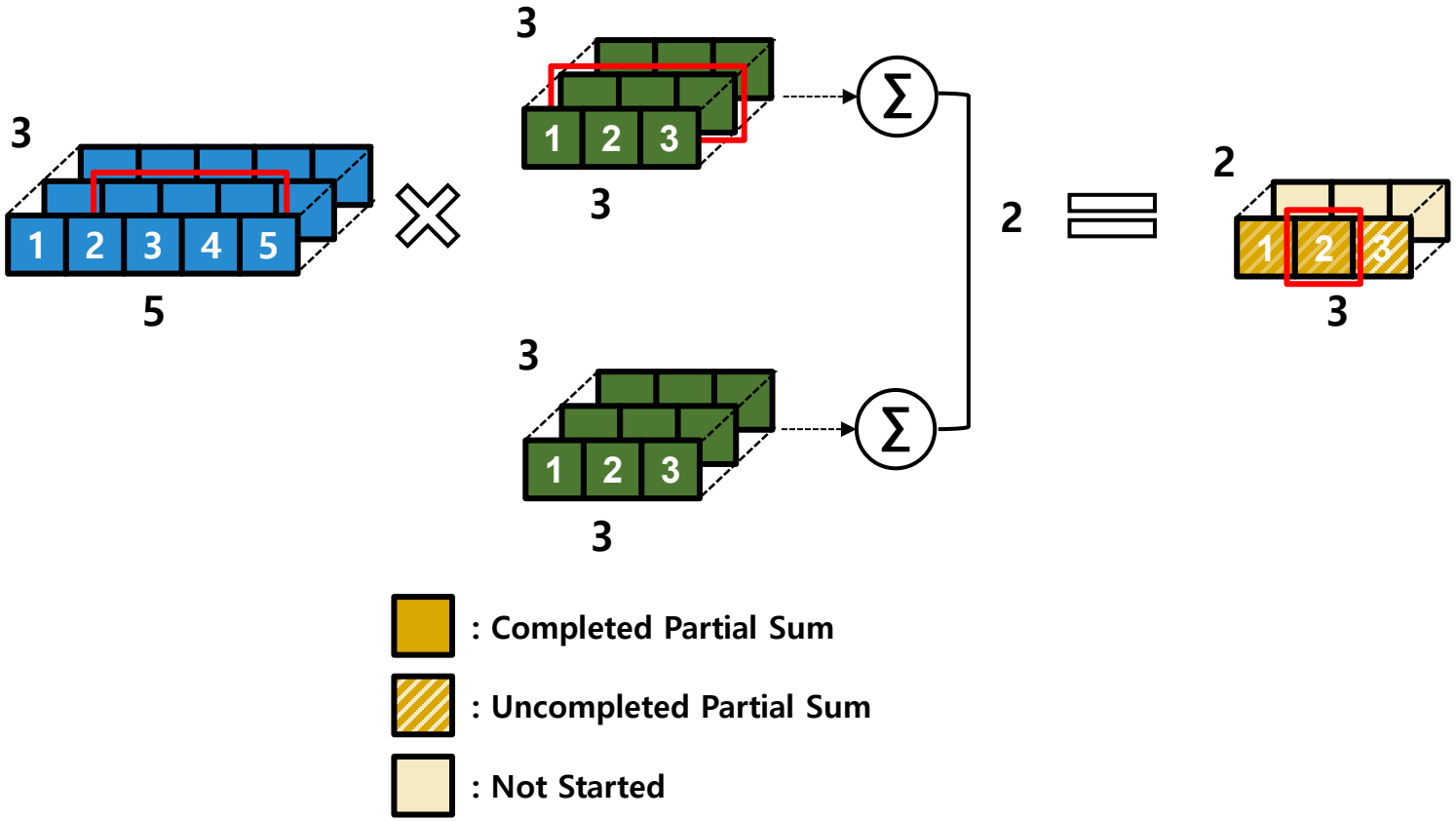


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (5)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

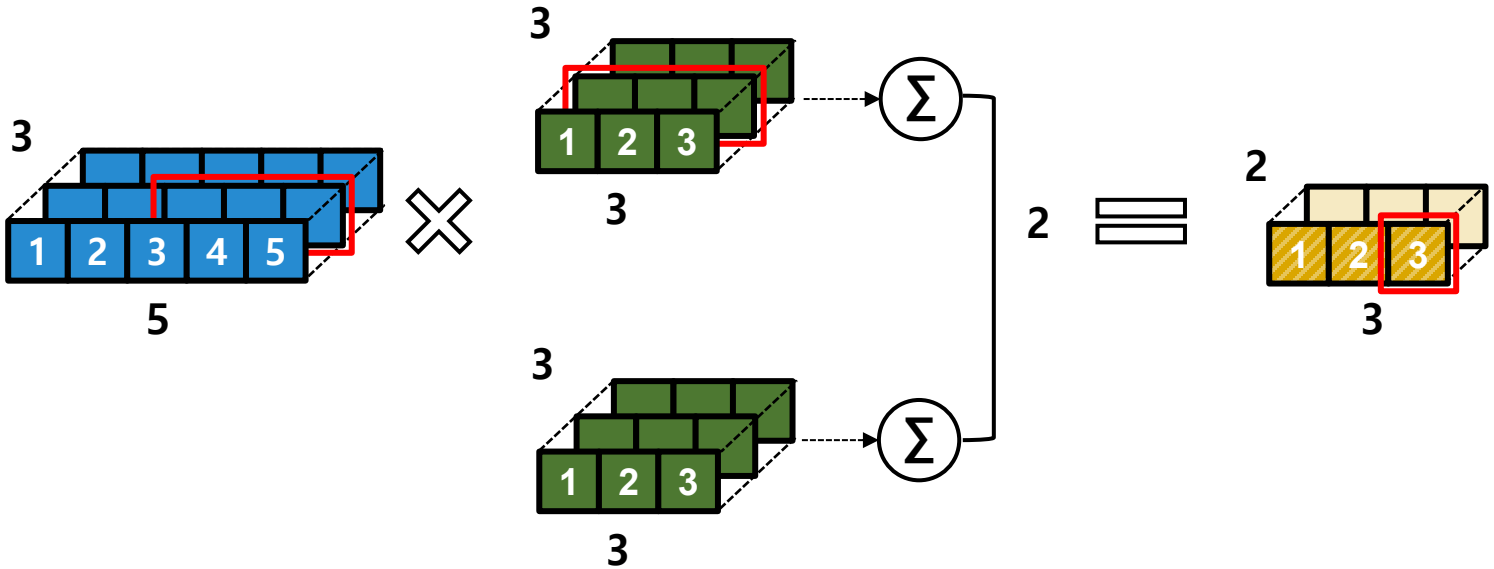


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (6)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**



: Completed Partial Sum

: Uncompleted Partial Sum

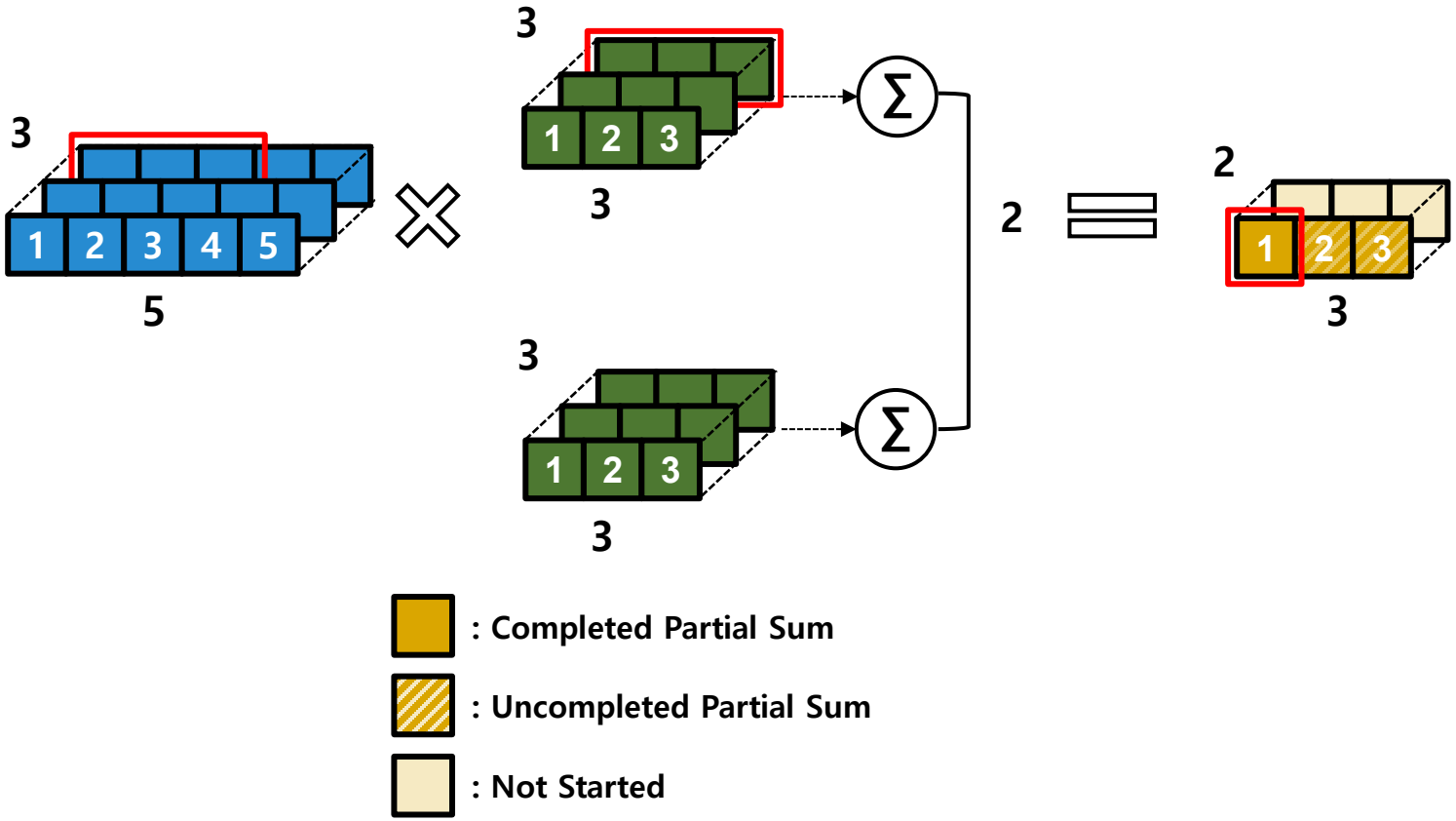
: Not Started

< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (7)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

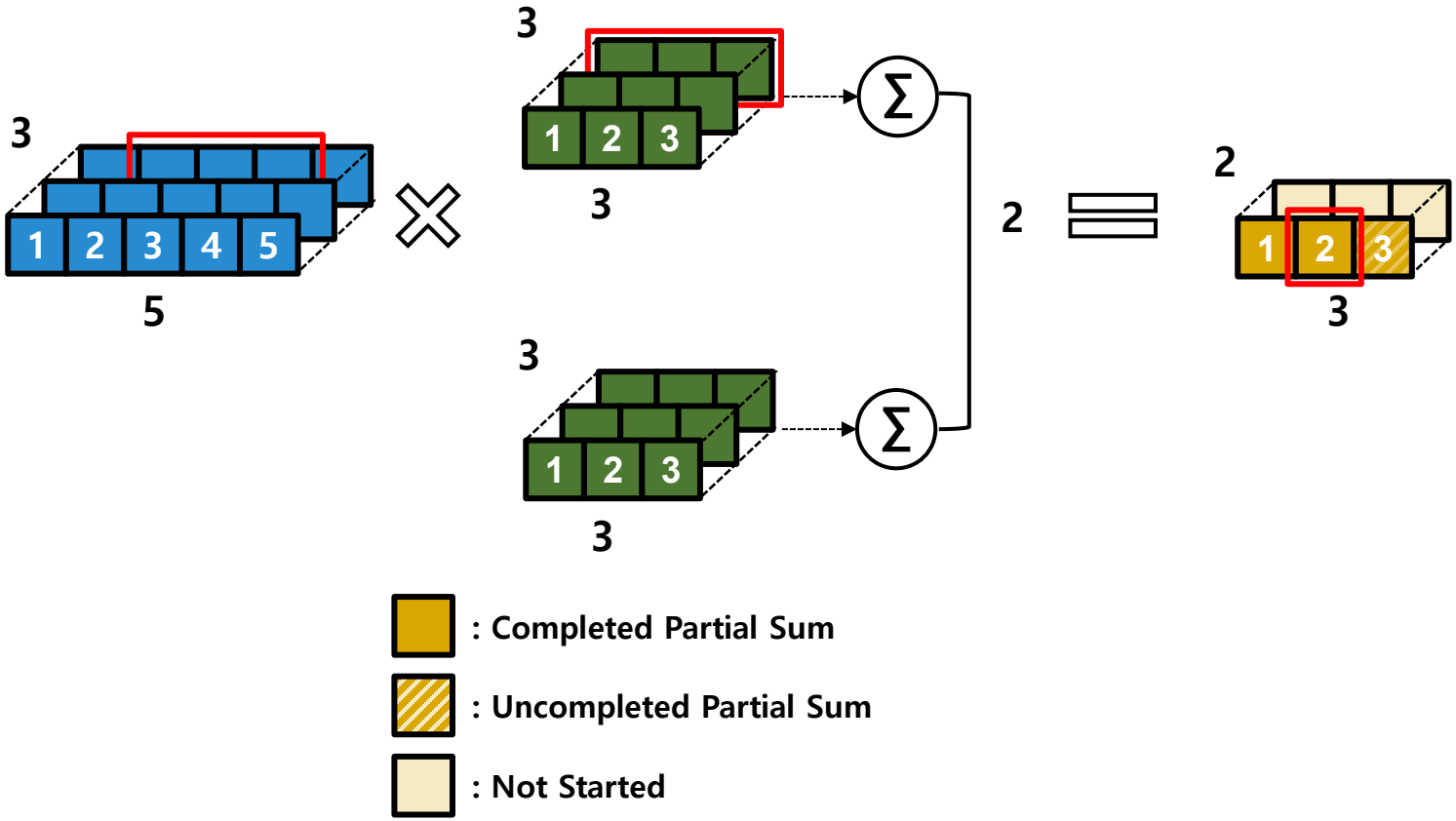


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (8)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

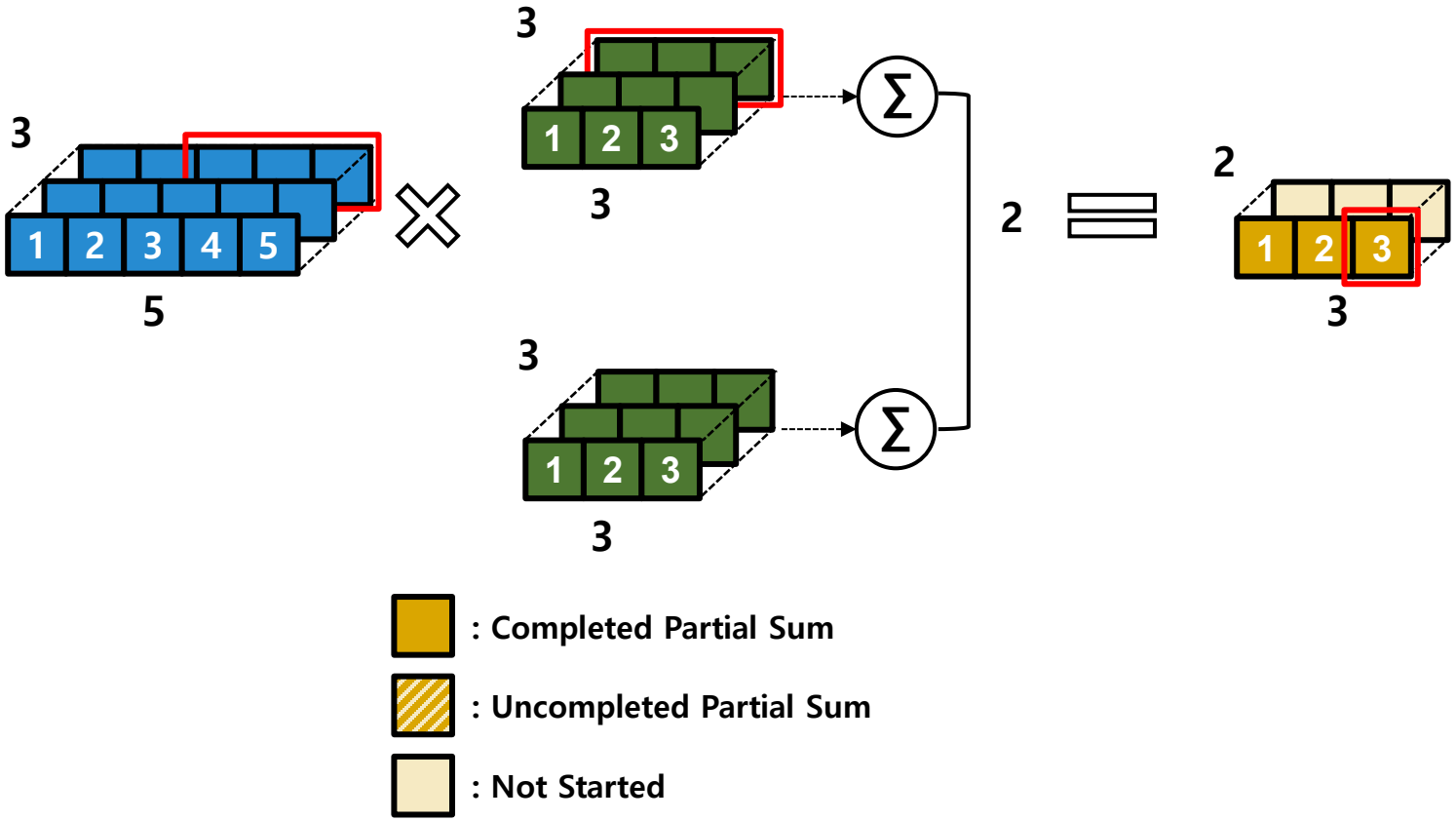


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (9)

Example Conv1d Operation (w/ 3 PEs)

- **Kernel-Level Parallelism**

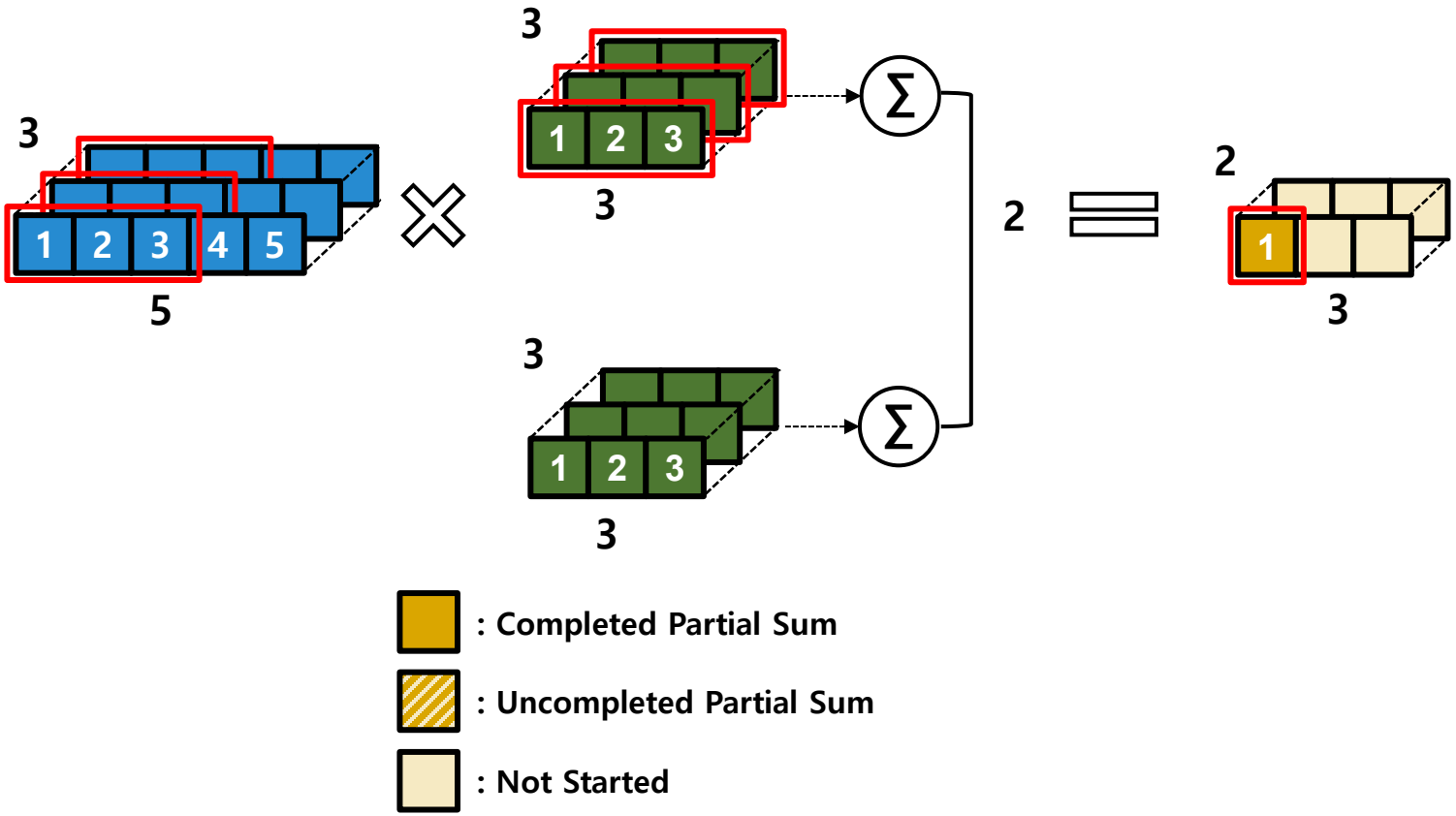


< Parallel Conv1d Execution (3 PEs) >

Conv1d Execution Flow (10)

Example Conv1d Operation (w/ 9 PEs)

- Channel × Kernel Parallelism

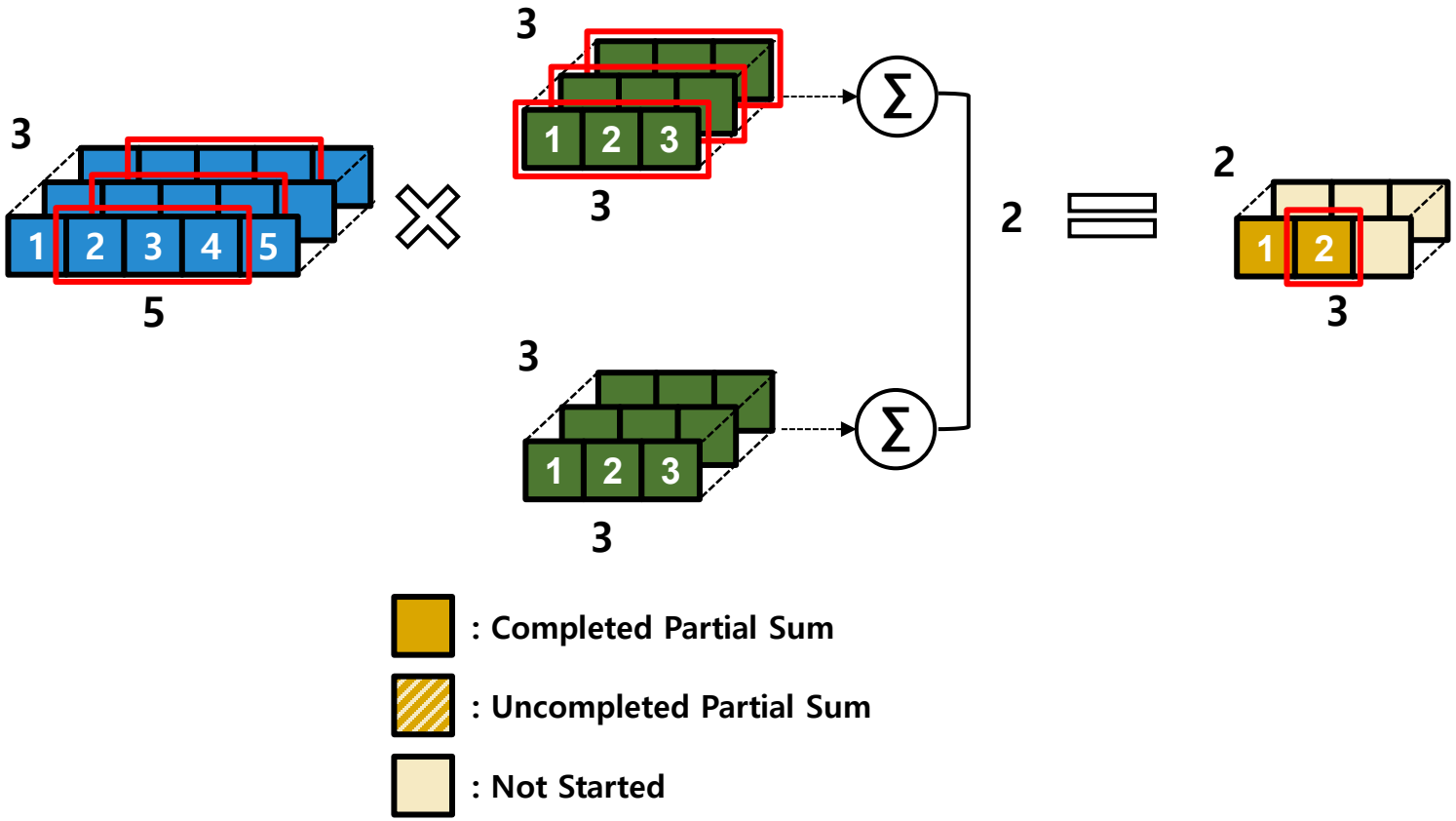


< Parallel Conv1d Execution (9 PEs) >

Conv1d Execution Flow (11)

Example Conv1d Operation (w/ 9 PEs)

- Channel × Kernel Parallelism

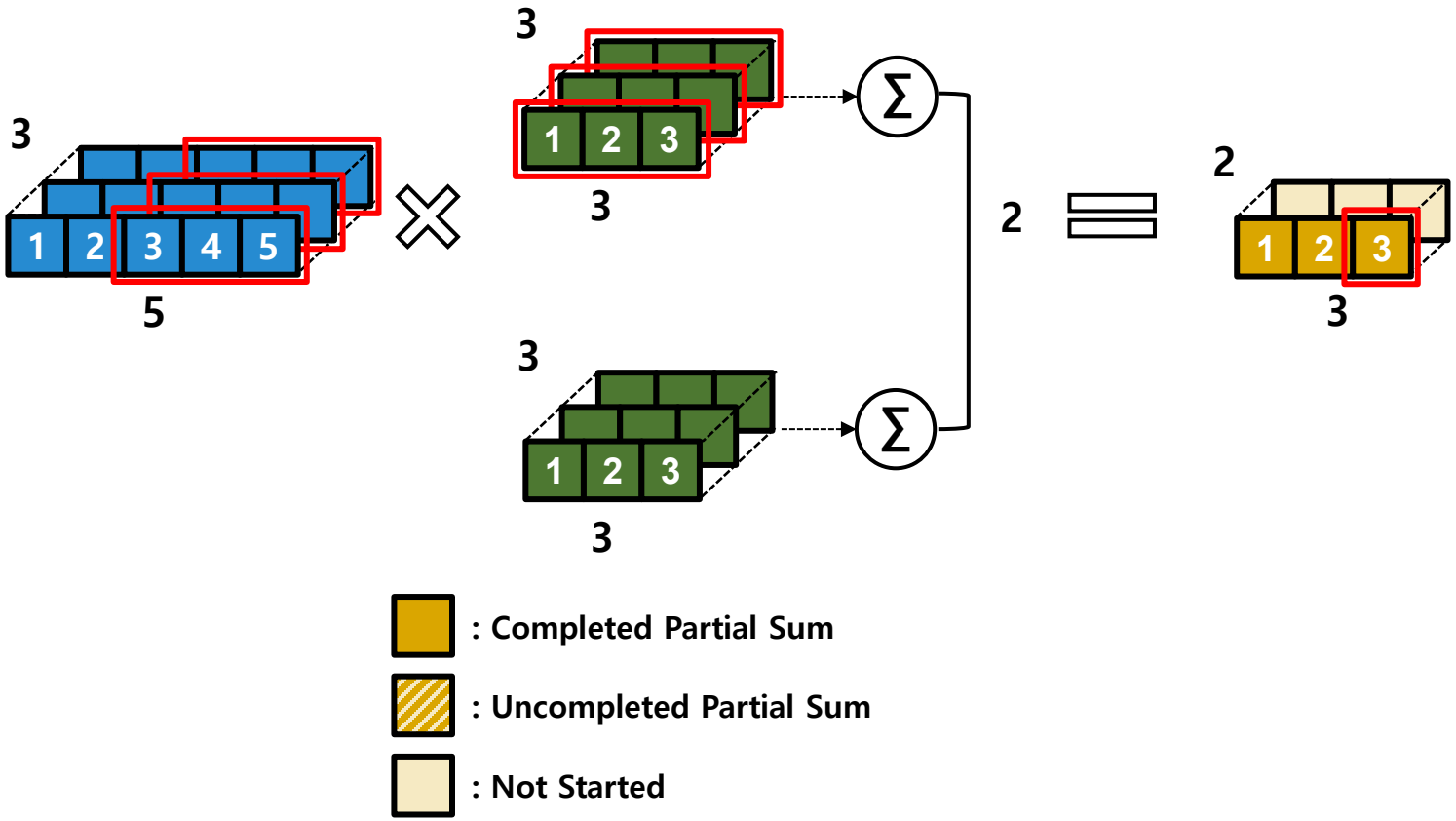


< Parallel Conv1d Execution (9 PEs) >

Conv1d Execution Flow (12)

Example Conv1d Operation (w/ 9 PEs)

- Channel × Kernel Parallelism



< Parallel Conv1d Execution (9 PEs) >

Thank You!